GateKeeper 기획서

Oct 21, 2026 · @은태현

개요

GateKeeper는 인프라 변경요청을 사내 변경관리 규정/서비스 구성정보(CMDB)/변경 일정/과거 이력과 대조해 위반과 위험을 찾고, 근거 조항을 붙여 승인/조건부/반려를 권고하는 에이전트다. 최종 결정은 변경관리자가 한다.

배경 (문제)

- 변경관리자는 매일 들어오는 변경요청을 직접 읽으며 규정 위반을 찾는다.
- 요청자는 자연어로 짧게 쓰고, 출발지 IP나 허용 기한 같은 필수 항목을 자주 빠뜨린다. 필수 항목은 변경 유형마다 다르다.
- 금요일 저녁 배포, 전체 대역 오픈 같은 위반이 사람의 눈을 지나치면 장애로 이어진다.

목표

- 확실하게 판정할 수 있는 규정 위반은 코드로 탐지하고, 근거 조항과 함께 승인자에게 전달한다.
- 승인자는 위험한 요청에만 집중한다. 사전 승인된 표준 변경을 뺀 모든 변경의 최종 승인은 사람이 한다.
- 정답셋 30건(평가용 20건)으로 버전별 판정 정확도를 비교해 효과를 수치로 보인다.

판단 기준의 원칙: 판정 기준은 항상 규정(법조문)이고, 과거 이력은 근거를 보강하는 참고(판례)다. 이력은 판정을 바꾸지 않고 위험도만 조정하며, 이력이 없어도 규정만으로 판단한다.

왜 규칙 엔진이 아니라 에이전트인가: 규칙 엔진은 정형 입력이 있어야 돈다. 실제 요청은 "주말 잠깐", "외부에서 접근 가능하게" 같은 자연어로 온다. LLM은 이를 구조화하고(①), 반려할 때 통과 가능한 조건부 대안을 만들고(④), 유사 사례를 요약한다(③). 판정은 코드가 맡아 일관성을 지키고, LLM은 사람이 하던 해석과 설명을 맡는다.

범위: 데모는 자체 화면으로 요청을 받는다. 실제 도입은 ITSM이나 그룹웨어 결재의 웹훅으로 요청을 받는 것을 전제로 한다. 승인 시간 준수, 기한 만료 후 닫기 같은 실행 단계 통제는 범위 밖이다.

페르소나

쓰는 사람(요청자), 결정하는 사람(승인자), 돈을 내는 사람(구매자)이 서로 다르다. 화면과 권한을 이 세 역할에 맞춰 나눈다. 아래 표로 먼저 훑고, 이어서 어떤 회사에 필요한지와 역할별 페인포인트를 장면으로 적는다.

페르소나	역할	상황	원하는 것	GateKeeper에서 쓰는 기능
김운영 (운영 엔지니어 3년차)	요청자	"이번 주말에 결제 DB 방화벽 잠깐 열어주세요"처럼 짧게 요청한다	무엇이 빠졌고 무엇이 위반인지 바로 알고 싶다	자연어 입력, 누락 항목 안내, 반려 사유와 조건부 대안
박CAB (변경 관리자)	승인자	매일 들어오는 변경요청을 직접 읽고 결재한다	위험한 요청만 근거와 함께 보고 결재하고 싶다	위반 조항/영향 범위/유사 사례 요약, 승인/반려 버튼
이팀장 (운영 팀장)	구매자	변경으로 인한 장애와 검토 리드타임을 책임진다	장애를 줄이고 검토 현황을 숫자로 보고 싶다	관리자 화면(판정 분포, 처리 시간), LangSmith 대시보드

이런 회사에 필요하다

변경이 잦고, 한 번의 실수가 매출이나 규제 문제로 바로 이어지는 회사일수록 효과가 크다.

회사 유형	변경이 위험해지는 순간	지금 겪는 문제	GateKeeper가 주는 것
결제대행(PG) / 핀테크	결제 파트너 연동용 방화벽 오픈, 정산 주간의 DB 작업	결제 DB 하나가 멈추면 모든 가맹점의 결제가 멈춘다. 감사 때마다 변경 승인 증적을 다시 모은다	근거 조항이 붙은 판정과 감사 로그가 그대로 증적이 된다
이커머스 / 게임	대형 할인 행사, 시즌 오픈 직전 배포	행사 기간 배포 금지 공지가 메신저로만 돌아 요청자가 모르고 올린다	변경 캘린더와 대조해 금지 기간 요청을 제출 단계에서 걸러낸다
IT 서비스 (운영 대행)	한 팀이 여러 고객사 인프라를 함께 운영할 때	고객사마다 규정과 금지 시간이 달라, 담당자가 바뀌면 이전 고객사 기준으로 판단한다	고객사별 규정 Resource와 CMDB MCP 서버만 바꿔 끼운다

기존 ITSM과의 관계: 이미 ITSM을 쓰는 회사에는 대체재가 아니라 그 앞단의 사전 검토로 붙는다. 규정이 사내 문서로만 있는 회사도 규정 Markdown과 MCP 서버만 준비하면 붙일 수 있고, 판정마다 근거 조항이 붙는다는 점이 차별점이다.

아래 페르소나는 이 중 결제대행사 A페이(가상)를 기준으로 쓴다. 같은 금요일 방화벽 요청 한 건이 세 사람을 차례로 지나간다. 장면 속 수치는 상황을 그리기 위한 가정이다.

김운영 / 요청자

장면: 목요일 오후 4시, 다음 주 월요일 오픈을 앞둔 신규 가맹점 파트너가 "테스트 서버에서 결제 DB에 붙어야 한다"고 연락한다. 김운영은 사내 메신저에 "금요일 저녁 8시에 payments-db 방화벽 좀 열어주세요. 외부 파트너사가 접속해야 해요"라고 남긴다.

페인포인트

- **규정을 제출하고 나서야 안다**: 변경관리 규정이 위키, 공지 메일, 회의록에 흩어져 있고 개정 이력도 따로 논다. 금요일 18시 이후 배포 금지(CHG-01), DB 포트 외부 노출 금지(CHG-03)를 모른 채 올렸다가 반려를 받고서야 안다.
- **반려 사유가 한 줄이라 다시 물어야 한다**: "규정 위반, 재작성 바람"이 돌아온다. 어느 조항인지, 무엇을 바꾸면 통과되는지 메신저로 다시 묻고 답을 기다린다.
- **반려 한 번이 일정을 며칠 민다**: 승인 회의가 주 2회(가정)라 반려되면 다음 회의까지 기다려야 한다. 파트너 오픈이 밀릴 것 같으면 "긴급 변경"으로 올려 검토를 건너뛰고 싶어진다.
- **지금의 우회책이 실수까지 복사한다**: 예전에 통과된 요청서를 복사해 고쳐 쓴다. 그 요청서에 없던 허용 기한은 이번에도 빠지고, "잠깐" 연 방화벽이 몇 달째 열려 있다.

GateKeeper가 바꾸는 것: 제출하는 순간 빠진 항목과 위반 조항을 보여주고, "파트너 IP 1개만 허용, 평일 14시, 허용 기한 7일" 같은 조건부 대안을 함께 준다. 반려가 회의를 기다리는 일이 아니라 바로 고쳐서 다시 내는 일이 된다.

성공 기준: 첫 제출에서 통과하는 비율, 요청 1건당 재제출 횟수

박CAB / 승인자

장면: 금요일 오전, 결재함에 변경요청 40건(가정)이 쌓여 있다. 대부분은 로그 레벨 변경이나 인증서 갱신 같은 반복 작업이고, 그 사이에 김운영의 결제 DB 방화벽 요청이 섞여 있다.

페인포인트

- **한 건을 보려고 화면 네 개를 연다**: 요청서, CMDB(이 서버가 Tier-1인지, 어떤 서비스가 기대는지), 변경 캘린더(같은 시간대의 다른 변경), 장애 이력(비슷한 변경으로 사고가 난 적 있는지)을 각각 다른 시스템에서 찾아 맞춰 본다.
- **무난한 39건이 위험한 1건을 가린다**: 대부분 문제가 없으니 승인 버튼에 손이 습관적으로 간다. 보안 관제의 경보 피로와 같은 구조로, 위험한 요청이 반복 작업 사이에 묻힌다.
- **사고가 나면 "왜 승인했나"를 혼자 설명한다**: 판단 근거가 메신저, 메일, 머릿속에 흩어져 있어 나중에 다시 맞추기 어렵다.
- **기준이 사람에게 붙어 있다**: 검토 품질이 담당자의 경험에 달려 있어, 휴가나 인수인계 때 같은 요청에 다른 판단이 나온다.

GateKeeper가 바꾸는 것: 표준 변경은 위반이 없으면 자동 승인되어 결재함에 올라오지 않고, 위험한 건은 위반 조항, 영향 범위, 유사 사례가 한 화면에 요약된 채로 올라온다. 승인 이유가 trace_id와 함께 감사 로그에 남는다.

성공 기준: 요청 1건당 검토 시간, 위반을 놓친 건수

이팀장 / 구매자

장면: 분기 보고에서 "이번 분기 장애 중 변경 때문에 난 것이 몇 건이냐"는 질문을 받는다. 같은 달에 ISMS-P 사후 심사가 잡혀 있어 변경 승인 증적도 모아야 한다.

페인포인트

- **변경이 큰 장애 원인인데 검토 인력은 못 늘린다**: 사람 실수로 난 큰 장애 대부분이 절차를 지키지 않았거나 절차 자체가 허술해서 생긴다(아래 사례). 그렇다고 변경관리자를 더 뽑을 예산은 없다.
- **심사 때마다 증적을 손으로 모은다**: ISMS-P 2.9.1은 변경 절차를 세우고 지키는지, 변경 전에 성능과 보안 영향을 분석했는지 확인한다. 결재는 그룹웨어, 영향 분석은 엑셀, 논의는 메신저에 있어 심사 전 며칠을 정리에 쓴다.
- **검토 현황을 숫자로 말할 수 없다**: 몇 건이 들어와 몇 건이 반려됐는지, 반려 사유 1위가 무엇인지, 검토에 얼마나 걸리는지 아무도 바로 답하지 못한다. 규정을 어디서 고치고 교육을 어디에 할지 근거가 없다.

GateKeeper가 바꾸는 것: 판정 분포, 반려 사유 순위, 처리 시간을 관리자 화면과 LangSmith에서 바로 본다. 감사 로그가 곧 심사 증적이다.

성공 기준: 변경 때문에 난 장애 건수, 심사 증적 준비 시간

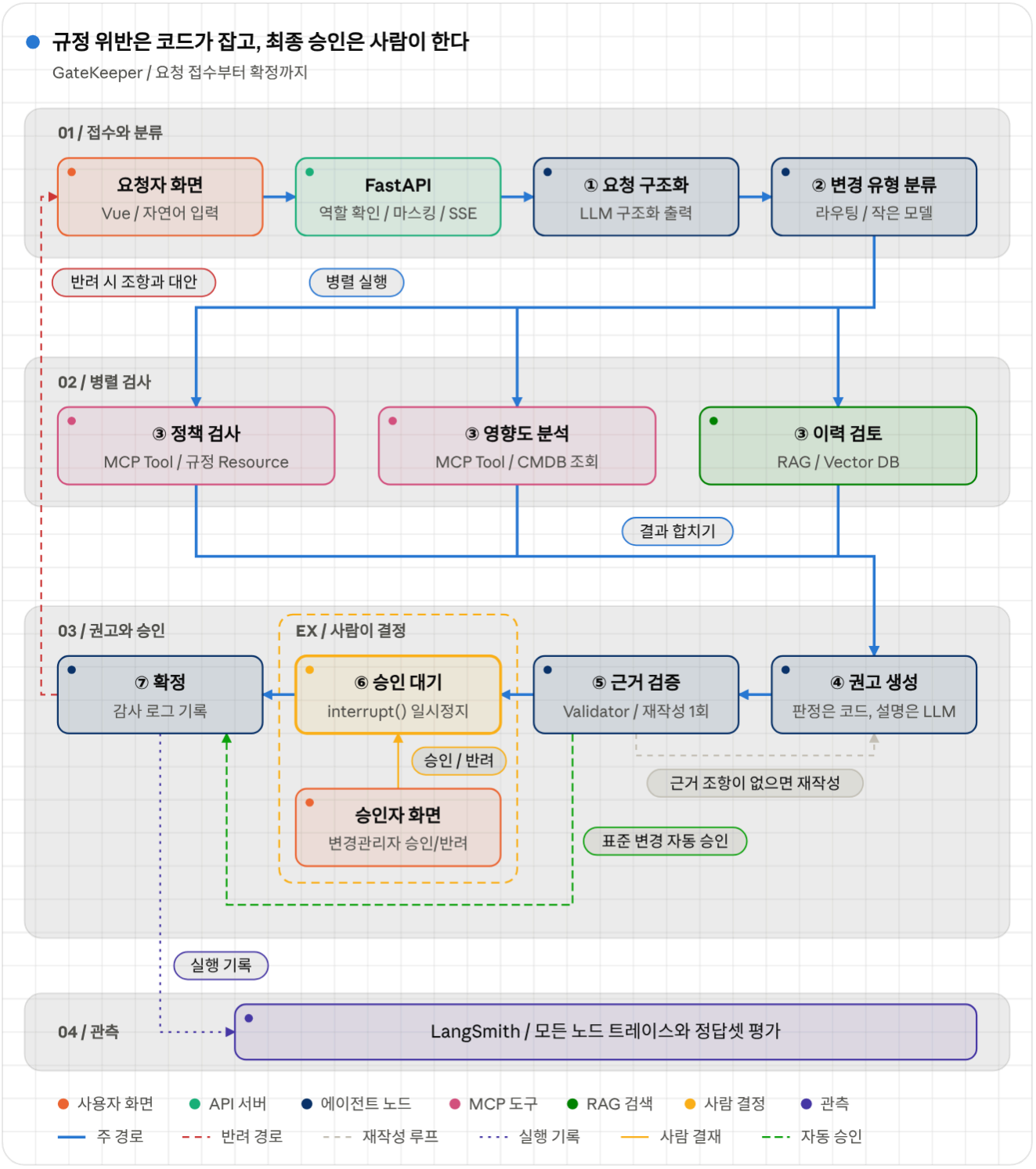
페인포인트의 근거가 된 실제 사례

- **KT 전국 통신 장애 (2021년 10월 25일)**: 라우터 교체 작업은 새벽 1시부터 6시까지로 승인됐지만 오전 11시께 진행됐고, 관리자는 자리를 비웠다. 명령어 스크립트를 두 단계로 검토했지만 exit 누락을 잡지 못해 약 89분간 전국 유무선 인터넷이 멈췄다. 정부는 이를 내부 체계가 무너진 인재로 봤다. (뉴스토마토, 이투데이)
- **Uptime Institute 연간 장애 분석 2025**: 지난 3년 사이 조직의 40% 가까이가 사람 실수로 큰 장애를 겪었고, 그중 85%는 절차를 지키지 않았거나 절차에 결함이 있어서 생겼다. IT와 네트워크 원인 장애는 2024년 23%로 늘었고, 복잡해진 환경의 변경관리 문제와 설정 오류가 원인으로 꼽혔다. (Business Wire)
- **ISMS-P 인증기준 2.9.1 변경관리**: 정보시스템 자산의 모든 변경내역을 관리하는 절차를 세워 지키고, 변경 전에 성능과 보안 영향을 분석하도록 요구한다. (ITWiki)

GateKeeper가 막을 수 있는 범위도 분명히 해 둔다. 승인된 시간과 다르게 실행하는 것 같은 실행 단계의 일탈은 막지 못한다. 대신 승인 단계의 누락(작업 시간대, 롤백 계획, 테스트 증적)을 규칙으로 잡고, 승인 내용을 기록으로 남겨 실제 실행과 대조할 기준을 만든다.

전체 흐름

요청은 구조화와 유형 분류를 거쳐 세 가지 검사를 병렬로 받고, 권고와 근거 검증 뒤 승인자의 결재를 기다린다.



GateKeeper 처리 흐름 / 7단계, 병렬 검사 3개, 사람 승인 1회

표준 변경(사전 승인된 저위험 요청)은 ②에서 정책 검사만 거치고, 위반이 없으면 초록 점선처럼 ⑥을 건너뛰어 자동 승인으로 ⑦에 기록된다. 위반이 하나라도 있으면 일반 변경처럼 사람에게 올라간다. 승인자가 반려하면 빨간 점선을 따라 요청자에게 위반 조항과 조건부 대안이 돌아간다. 상자 색은 일을 맡은 쪽(화면, API 서버, 에이전트 노드, MCP 도구, RAG, 사람)을 뜻한다.

기능별 기술 설계

기술마다 맡는 일이 분명하다. 확실하게 판정해야 하는 것(시간, 대역, 포트, 권한)은 코드와 MCP Tool, 해석과 설명은 LLM, 비슷한 사례 찾기는 RAG, 책임은 사람(HITL)이 맡는다. 예시 요청은 "이번 주 금요일 저녁 8시에 payments-db 방화벽 좀 열어주세요. 외부 파트너사가 접속해야 해요"다.

F1. 요청 구조화와 누락 검출

- **상황**: 자연어 요청을 "대상 payments-db / 유형 방화벽 오픈 / 시간 금 20:00 / 출발지 IP 미지정 / 허용 기한 없음"으로 바꾼다.
- **기술**: LLM 구조화 출력(Pydantic 스키마 강제) + Pydantic 검증
- **이유**: 자연어 해석은 LLM만 할 수 있다. 형식과 필수 항목 누락은 코드가 확실하게 판정한다.
- **검토한 대안**: 고정 입력 폼(정확하지만 요청자가 번거롭게 여김), 정규식 파싱(표현이 다양해 실패)

F2. 변경 유형 라우팅

- **상황**: 로그 레벨 변경 같은 사전 승인된 저위험 요청까지 전체 검토를 돌리지 않는다.
- **기술**: Query Routing(작은 모델 + 구조화 출력) + LangGraph `add_conditional_edges`
- **이유**: 모든 요청에 전체 검토는 시간과 비용 낭비다. 애매하면 일반 변경(전체 검토)으로 보내는 fail-safe로 설계한다. 유형에 따라 필수 항목도 달라진다(방화벽은 출발지 IP와 허용 기한, 배포는 롤백 계획과 테스트 증적).
- **검토한 대안**: 항상 전체 검토(느림), 요청자가 유형 선택(잘못 고르면 검토를 우회)

F3. 규정 위반 탐지

- **상황**: "금 20:00"은 배포 금지 시간대, "DB 포트 외부 노출"은 금지 조항 위반이다.
- **기술**: MCP Tool(시간대/CIDR/포트/필수 항목을 검사하는 결정론 코드) + MCP Resource(`rules://CHG-01` 형태의 규정 원문)
- **이유**: 시간과 대역 계산은 LLM이 틀리기 쉽고 코드는 항상 같은 답을 낸다. 규정은 잘 바뀌지 않는 정적 문서라 Resource가 맞다.
- **검토한 대안**: LLM 단독 판정(비결정적, 근거 추적 불가), 규정 전체를 RAG로 검색(규정 10개 안팎이라 과설계)

F4. 영향도 분석

- **상황**: payments-db는 Tier-1이고 payments/billing 서비스가 의존한다. 같은 변경도 테스트 서버면 저위험, 결제 DB면 고위험이다.
- **기술**: MCP Tool(CMDB 조회: 등급, 담당 팀, 의존 서비스) + 변경 캘린더 조회(배포 금지 기간, 동시간 변경)
- **이유**: 서비스 상태와 일정은 시점마다 달라지는 동적 데이터라 Tool이 맞다. 고객사마다 CMDB가 달라 MCP 서버만 교체하면 연동된다.

- **검토한 대안**: 코드 내부 함수(고객사마다 코드 수정), 별도 REST 마이크로서비스(이 규모에서 과설계)

F5. 과거 이력 검토

- **상황**: "파트너 접속용 방화벽 변경 후 결제 지연 장애" 같은 유사 사례를 찾아 근거를 보강한다.
- **기술**: RAG(과거 변경/장애 이력 100건, 벡터 DB, 유사도 임계값)
- **이유**: 표현이 달라도 비슷한 사례를 찾으려면 의미 검색이 필요하다. 판정(승인/조건부/반려)은 바꾸지 않는다. 같은 서비스에 유사 장애 이력이 있으면 위험도를 한 단계 올려 승인자 결재함 맨 위로 올리고 "유사 장애 있음"을 표시한다.
- **검토한 대안**: 키워드 검색(표현이 다르면 놓침), 이력 전체를 프롬프트에 넣기(길이와 비용 증가)

F6. 검사 3종 병렬 실행 (멀티에이전트)

- **상황**: 정책 검사/영향도 분석/이력 검토를 동시에 실행하고 결과를 합친다.
- **기술**: LangGraph 서브그래프(compiled subgraph) 3개를 병렬 실행(fan-out/fan-in). 정책/영향도/이력 전문 에이전트가 각자 Tool을 부르고, LLM으로 결과를 해석해 근거 요약을 만든다. 판정 값 자체는 Tool이 내고, 메인 그래프가 세 에이전트 호출을 강제한다.
- **이유**: 세 검사는 서로 독립이라 병렬로 지연을 줄인다. 필수 검사를 LLM 재량에 맡기면 빠뜨릴 수 있어 코드가 강제한다(권위 게이트).
- **검토한 대안**: 단일 ReAct 에이전트(LLM이 검사를 빠뜨릴 수 있음), Supervisor가 LLM으로 계획(실행할 때마다 계획이 달라짐)

F7. 권고 생성과 근거 검증

- **상황**: 판정 기준(아래 표)으로 판정과 위험도를 정해 "반려 권고 + 조건부 대안(파트너 IP 1개만, 평일 낮, 허용 기한 7일 지정 시 재검토)"을 만들고, 모든 반려 사유에 실제 규정 조항이 붙었는지 확인한다.
- **기술**: 판정과 위험도는 코드, 설명은 LLM + Validator(rule_id 존재 검사 + LLM-as-Judge), 재작성 1회 한도(루프 가드)
- **이유**: 판정은 일관성이 필요하고, LLM이 없는 규정을 지어내는 환각을 막아야 한다.
- **검토한 대안**: LLM이 판정까지(실행마다 달라짐), 검증 없이 출력(없는 조항을 인용할 위험)

판정 기준

판정은 규정 위반과 누락으로만 정한다. 위험도는 판정을 바꾸지 않고 결재 순서와 강조에만 쓴다.

구분	조건	예시
반려 권고	금지 조항 위반이 1개 이상 (CHG-01 배포 금지 시간대, CHG-02 전체 대역 오픈, CHG-03 DB 포트 외부 노출, CHG-06 동시간 변경, CHG-10 배포 동결 기간). 재제출이 필요하고 조건부 대안을 함께 준다	금요일 20시, 출발지 미지정으로 payments-db 5432 오픈
조건부 승인 권고	금지 위반은 없고 보완하면 되는 누락이 있음 (CHG-04 롤백 계획, CHG-05 사전 공지, CHG-08 테스트 증적, CHG-09 허용 기한). 조건을 달아 승인한다	평일 14시, 파트너 IP /32 지정, 허용 기한 없음 → 기한 7일 조건
승인 권고	위반과 누락이 없음	평일 14시, 파트너 IP /32, 허용 기한 7일
위험도 (별도 축)	CMDB 등급(Tier-1 높음, Tier-2 중간, 그 외 낮음). 같은 서비스에 유사 장애 이력이 있으면 한 단계 올린다	결재함 정렬, "유사 장애 있음" 표시

F8. 사람 승인과 감사 로그

- **상황**: 일반 변경이면 그래프가 멈추고 박CAB의 결재를 기다린다. 표준 변경은 정책 위반이 없으면 ⑥을 건너뛰고 자동 승인으로 기록된다. 승인 버튼은 승인자에게만 보인다.
- **기술**: LangGraph `interrupt()` + SQLite checkpointer(팩토리 함수로 생성) + FastAPI `Depends` 역할 확인 + trace_id가 담긴 구조화 감사 로그
- **이유**: 고객사 시스템에 손대는 결정은 책임 소재가 필요하다. 판단부터 승인까지 하나의 흐름으로 기록되고, 서버가 재시작돼도 대기 중인 요청이 유지된다.
- **검토한 대안**: 에이전트 자동 승인(책임 소재 없음), 메일로 승인(이력이 따로 놈), MemorySaver(재시작하면 대기 중인 요청 유실)

F9. 화면과 실시간 진행 표시

- **상황**: "정책 검사 완료, 영향도 분석 중"처럼 진행 상황을 화면에 보여주고, 요청 본문의 비밀값은 LLM에 넘기기 전에 가린다. 승인은 몇 시간 뒤에 날 수 있어 SSE는 ⑥ 승인 대기에서 닫고, 결과는 상태 조회 API(GET /requests/{id})로 받는다.
- **기술**: Vue + FastAPI + SSE, lifespan에서 그래프와 MCP 클라이언트 1회 생성, 비밀값 마스킹 미들웨어, CORS 설정, 요청 상태 조회 API
- **이유**: 서버에서 화면으로 가는 단방향 알림이면 SSE로 충분하다. 요청마다 그래프를 만들면 MCP 프로세스가 계속 늘어난다.
- **검토한 대안**: 폴링(지연과 낭비), WebSocket(양방향이 필요 없어 과설계)

F10. 관측과 평가

- **상황**: 관리자가 "왜 반려됐는지" 사후에 추적하고, 버전별 판정 정확도를 비교한다.
- **기술**: LangSmith 트레이싱 + Dataset/`evaluate()` + pytest 단위 테스트(결정론 Tool)
- **이유**: 노드별 입력/출력/시간이 트리로 남고, 같은 정답셋으로 실험끼리 나란히 비교된다.
- **검토한 대안**: 로그 파일만 남기기(요청 흐름을 재구성하기 어려움)

예외 상황 처리

어느 한 단계가 실패해도 전체 검토가 멈추지 않고, 확인하지 못한 부분은 "확인 불가"로 표시해 승인자에게 넘긴다.

상황	기술	동작
요청 본문에 DB 비밀번호나 키가 섞임	입력 마스킹(정규식 미들웨어)	LLM과 LangSmith로 넘어가기 전에 `****`로 치환
로그 레벨 변경 같은 사전 승인된 저위험 변경	라우팅 빠른 경로	정책 검사만 하고 영향도/이력 검사는 생략, 위반이 없으면 자동 승인
CMDB나 캘린더 조회가 타임아웃	with_retry(지수 백오프) + 부분 결과	재시도 후에도 실패하면 "영향도: 확인 불가"로 표시하고 나머지 결과로 진행
유사 이력이 없음	RAG 유사도 임계값	억지로 끌어오지 않고 "유사 사례 없음"으로 표시. 판단은 규정 기준이라 지장 없음
반려 후 요청자가 "허용 기한 넣었어요"라고 재요청	thread_id + checkpointer	처음부터 다시 입력하지 않고 이전 요청에 추가 정보를 반영해 재검토
Validator가 2번 연속 불합격	루프 가드(최대 재작성 횟수)	마지막 결과에 "근거 검증 미완료" 경고를 붙여 승인자에게 전달
요청자가 승인 API를 직접 호출	FastAPI `Depends` 역할 확인	403으로 차단하고 감사 로그에 기록
LLM 호출 실패나 토큰 한도 초과	에러 메시지 정제	내부 사정을 노출하지 않고 "자동 검토를 완료하지 못해 수동 검토로 넘깁니다"로 안내
요청자가 "긴급"으로 올려 검토를 우회하려 함	긴급 판정 조건(장애 티켓 번호 필수)	티켓 번호가 없으면 일반 변경으로 처리. 있고 열린 장애면 CHG-01, CHG-05, CHG-10을 면제하고 나머지 검사는 그대로 하며, 사후 보고 대상(CHG-07)으로 표시

상황	기술	동작
승인 대기 중 요청자가 화면을 닫음	상태 조회 API + checkpointer	다시 열면 상태 조회 API로 현재 단계와 결과를 받는다

개선 포인트와 Before/After 측정 계획

같은 평가용 정답 변경요청 20건에 세 버전을 차례로 돌려, 개선마다 판정 정확도와 근거 품질이 얼마나 바뀌는지 LangSmith 실험으로 비교한다.

버전	구성	더해진 개선
v0 (기준)	LLM 단독 판정. 규정 전문, CMDB, 변경 캘린더, 장애 티켓을 프롬프트에 넣고 판정까지 LLM이 함 (정보는 같고 구조만 다름)	—
v1	규정 검사를 MCP Tool(코드)로 분리 + 검사 3종 병렬 + 라우팅	판정 정확도, 검토 시간
v2	v1 + 이력 RAG + Validator	근거 인용 품질, 환각 차단

측정 지표

- 판정 정확도: 승인/조건부/반려가 정답과 같은 비율
- 구조화 정확도: ①이 뽑은 필드(대상, 시간, 출발지 IP, 포트, 기한)가 정답과 같은 비율. 판정이 틀렸을 때 구조화에서 틀렸는지 판정에서 틀렸는지 가른다
- 위반 조항 재현율/정밀도: 정답 rule_id를 빠짐없이, 틀린 조항 없이 찾았는가
- 근거 인용률: 반려 사유 중 실제로 존재하는 조항이 붙은 비율
- 평균 검토 시간: 요청 1건당 종료까지 걸린 시간
- 토큰 사용량: 라우팅과 작은 모델 분리 전후 비교
- 비밀값 노출 건수: 마스킹 전후 LLM 입력에 비밀값이 남은 건수

v0이 이미 너무 잘 맞으면 개선 폭을 보여주기 어렵다. Day1에 샘플 10건으로 v0을 먼저 돌려 보고, 정확도가 너무 높으면 "주말 잠깐", "외부에서 접근 가능하게" 같은 애매한 표현을 늘린다.

정답셋이 순환하지 않게: 규정, Tool 코드, 정답을 모두 직접 만들면 v1이 정답을 외운 것처럼 높게 나올 수 있다. 그래서 정답 라벨은 Tool 코드를 짜기 전에 규정 문서와 판정 기준만 보고 단다. 30건을 개발용 10건과 평가용 20건으로 나누고, 평가용은 개발 중에 열어보지 않는다. 가능하면 동료 1명이 평가용 라벨을 교차 검수한다. 20건이면 1건이 5%p라서 결과는 비율과 함께 건수로도 적는다.

데이터 준비

모든 데이터는 가상으로 직접 만든다. 규정과 정답을 내가 정하기 때문에 도메인 지식 부담이 작고, 정답 라벨만 직접 검수하면 된다.

데이터	형식	양	쓰는 곳	만드는 방법
변경관리 규정	Markdown (조항마다 rule_id)	10개 안팎	MCP Resource, 결정론 Tool	직접 작성
CMDB	JSON (등급, 담당 팀, 의존 관계)	서비스 16개 + 등록 파트너 3곳	MCP Tool (영향도 분석)	생성형 AI 초안 + 검수
변경 캘린더	JSON (배포 금지 기간, 예정된 변경)	2주치	MCP Tool (일정 충돌)	직접 작성
과거 변경/장애 이력	JSON/Markdown	100건 안팎	RAG (벡터 DB)	생성형 AI 초안 + 검수
정답 변경요청	JSON (본문, 정답 판정, 위반 rule_id)	30건	LangSmith Dataset (개발 10건 / 평가 20건)	생성형 AI 초안 + **라벨 직접 검수**
장애 티켓	JSON (티켓, 상태, 대상 서비스)	3건 (열림 2, 종료 1)	MCP Tool (긴급 변경 검증)	직접 작성

규정 초안 (예시)

1. CHG-01 배포 금지 시간대: 금요일 18시 이후 ~ 월요일 09시 이전에는 일반 변경을 하지 않는다.
2. CHG-02 전체 대역 오픈 금지: 방화벽 규칙에 0.0.0.0/0을 허용하지 않는다. 출발지를 적지 않았거나 "아무 데서나"처럼 범위가 없는 요청도 0.0.0.0/0으로 본다.
3. CHG-03 DB 포트 외부 노출 금지: 5432, 3306 포트는 사내망에서만 연다. 외부 파트너는 등록된 파트너 IP를 /32로 지정할 때만 예외로 한다.
4. CHG-04 롤백 계획 필수: 배포와 DB 변경은 롤백 절차를 적는다.
5. CHG-05 Tier-1 사전 공지: Tier-1 서비스 변경은 24시간 전에 공지한다.
6. CHG-06 동시간 변경 금지: 같은 서비스에 같은 시간대 변경을 겹치지 않는다.
7. CHG-07 긴급 변경 사후 보고: 긴급 변경은 장애 티켓 번호를 적고, 신속 검토 후 실행하며 24시간 안에 보고한다. 티켓이 열려 있고 대상이 그 장애 범위에 들면 CHG-01, CHG-05, CHG-10을 면제한다.
8. CHG-08 테스트 증적: 운영 반영 전 스테이징 테스트 결과를 첨부한다.
9. CHG-09 임시 오픈 만료일 필수: 외부 접근을 여는 방화벽 변경은 허용 기한(최대 30일)을 적는다.
10. CHG-10 배포 동결 기간: 변경 캘린더에 등록된 동결 기간(월말 정산, 대형 행사)에는 그 범위의 서비스를 변경하지 않는다. 표준 변경에도 적용한다.

변경 유형별 필수 항목

변경 유형	필수 항목	근거 조항
방화벽 / 접근 허용	출발지 IP, 대상과 포트, 허용 기한, 사유	CHG-02, CHG-03, CHG-09
애플리케이션 배포	작업 시간대, 롤백 계획, 스테이징 테스트 결과	CHG-01, CHG-04, CHG-08
DB 변경 (스키마, 설정)	롤백 계획(백업 포함), 테스트 결과, Tier-1 이면 사전 공지	CHG-04, CHG-05, CHG-08
표준 변경 (로그 레벨, 인증서 갱신)	대상, 작업 시간	사전 승인된 유형 목록, CHG-06, CHG-10

정답 변경요청 30건은 위반 없음, 조항 1개 위반, 여러 개 위반, 애매한 표현("주말 잠깐")이 고루 섞이게 만든다.

제작 결과 (10/21): `gatekeeper_data` 폴더에 규정 원문, CMDB, 변경 캘린더, 장애 티켓, 이력 100건(장애 15, 실패 후 롤백 12, 성공 73), 정답 요청 30건을 만들었다. 라벨은 규정 문서와 판정 기준만 보고 붙였고, 규정을 그대로 옮긴 별도 검수 스크립트로 30건 모두 일치를 확인했다. 평가용 20건에는 금지/보완 조항 9개가 모두 들어 있다.

평가 항목 매핑, 일정, 리스크

평가표의 기술 요소 일곱 개가 모두 기능에 자연스럽게 들어간다. 가장 큰 리스크는 데이터 작성량과 사람 승인(HITL) 구현이다.

평가 기술 요소	GateKeeper에서 들어가는 곳
Multi-Agent	정책/영향도/이력 전문 서브에이전트 3개 병렬 + Validator (F6, F7)
MCP	CMDB/변경 캘린더 Tool, 규정 Resource (F3, F4)
RAG	과거 변경/장애 이력 검색 (F5)
의존성 주입	FastAPI `Depends` 역할 확인, checkpointer 팩토리 (F8)
LangSmith	노드별 트레이스 + 정답셋 `evaluate()` (F10)
모듈화	`agent/nodes/`, `mcp_server/`, `persistence/`, `api/` 분리
단위 테스트	결정론 Tool(시간대/CIDR/포트 검사) pytest (F3)

일정

1. Day1 (10/20): 주제 확정, 기획, 시트 제출, 데이터 작성(규정/CMDB/캘린더/정답 30건/이력 100건), v0 샘플 판정
2. Day2 (10/21): 그래프 노드, MCP 서버, HITL, API/SSE, 화면 수정
3. Day3 (10/22) 오전: LangSmith 평가, Before/After 수치, 보고서/발표 자료, **13시 제출**

Day2 잘라낼 순서 (막히면 위에서부터 줄인다)

- 화면: Base 프로젝트의 Vue 화면을 그대로 쓰고, 승인자 화면은 승인/반려 버튼만 둔다
- RAG: 이력을 50건으로 줄이고 로컬 Vector DB 하나로 끝낸다
- HITL: interrupt가 막히면 "승인 API로 상태 변경" 방식으로 축소한다

끝까지 지키는 것은 결정론 Tool과 단위 테스트, 병렬 검사, 판정 기준, 정답셋 평가다. 배점이 큰 기능 완성도, 기술 요소, Before/After가 여기서 나온다.

리스크와 트레이드오프

항목	내용	대응
규정이 두 곳에 존재	코드(Tool)와 규정 문서가 어긋날 수 있음	rule_id로 연결하고 단위 테스트로 고정
그래프 유연성 저하	필수 검사를 코드로 강제해서, 새 검사를 추가하려면 그래프를 수정해야 함	검사 노드를 모듈로 분리해 추가 비용을 줄임
RAG 오탐	유사하지 않은 사례가 끌려옴, 임베딩 비용과 지연 증가	유사도 임계값, 이력은 참고로만 반영
단일 머신 전제	MCP subprocess 방식이라 서버를 여러 대로 늘리기 어려움	재검토 조건: 여러 백엔드가 공유해야 하면 원격 MCP로 전환
데이터 작성량	Day1에 만들 것이 많음	생성형 AI로 초안, 정답 라벨만 직접 검수
HITL 구현 난이도	interrupt와 checkpointer는 처음 다루는 부분	막히면 "승인 API로 상태 변경" 방식으로 축소
로컬 모델 속도	qwen3:8b로 30건 반복 평가는 오래 걸림	개발은 Ollama, 최종 평가는 Claude API(Sonnet 5.5, 라우팅은 Haiku 5.5)로 3회 반복
표준 변경 자동 승인	표준 변경 목록이 틀리면 위험한 변경이 사람 없이 통과함	목록은 CAB가 관리하고, 자동 승인 건도 감사 로그에 남겨 주 1회 샘플 검토
규정 밖 위험	판정을 코드가 하므로 규정에 없는 새 위험은 잡지 못함	LLM이 "규정 외 우려" 메모를 승인자 화면에만 표시하고 판정에는 반영하지 않음